

DL for ENG

LECTURE 8.2

Transient Heat

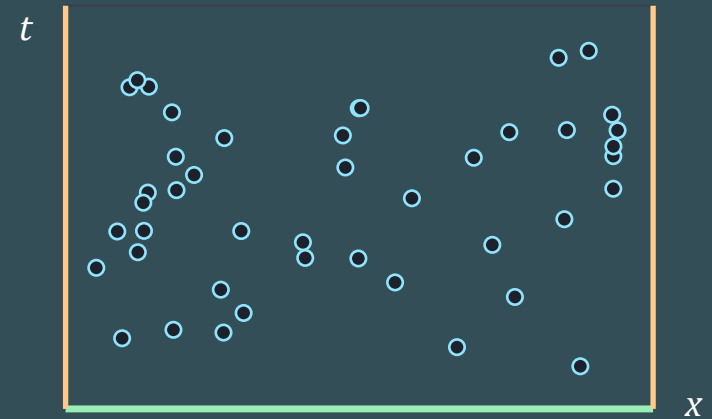
Remus Teodorescu

*With support from Research Assistant Noman Khan -
nomank@energy.aau.dk*

Aalborg University · ret@et.aau.dk

Deep Learning for Engineering · 2026

TIME · CAUSALITY · THE INVERSE PROBLEM



green: the field at $t = 0$ · amber: the walls

Time is one more input, and the whole block of space and time is trained at once. That buys a solution readable at any instant, and it costs the arrow of time: the loss does not know that early instants come first. The same plate then gives up its diffusivity to a few sensors.

Learning Objectives

SECTIONS, IN ORDER

- 1 **Time** — the transient equation, the space–time block, the initial field
- 2 **Causality** — why the loss does not know the arrow of time, and four fixes
- 3 **Plate and inverse** — the plate in time, checking it, and recovering α

READ, AND WHAT EACH ANSWERS

Liu ch. 4.1 – 4.3 what does time add to the plate?

Liu ch. 4.4 why are the early times fitted last?

Liu ch. 4.6 how is a transient checked, and inverted?

Transient Conduction

Transient conduction — the heat equation with units:

$\partial T / \partial t = \alpha \nabla^2 T + Q / (\rho c_p)$, where $\alpha = k / \rho c_p$ is the diffusivity. It is parabolic, so besides the walls it needs the field at $t = 0$, and its time constant $\tau = L^2 / (d \pi^2 \alpha)$ says how long the transient lasts.

ONE NEW TERM, ONE NEW CONSTANT

$$\frac{\partial T}{\partial t} = \alpha (T_{xx} + T_{yy}) + \frac{Q}{\rho c_p} \quad \alpha = \frac{k}{\rho c_p} \text{ [m}^2/\text{s]}$$

$$\tau = \frac{L^2}{d \pi^2 \alpha}$$

d dimensions: the square decays twice as fast as a slab of the same size

THE TRANSIENT IN PYTORCH

```
xyt = torch.cat([x, y, t], 1)          # three inputs now
T = net(xyt)                           # the field in space and time
F = grad(T, t) - alpha * lap(T, xy) - Q_c # the residual; grad = autograd
tau = L**2 / (2 * pi**2 * alpha)        # the square's time constant
```

THE BENCHMARK MODE DECAYS



Ex_08.2's benchmark: one mode, decaying and keeping its shape. Amber is τ , orange 4τ , where it has settled. At $t = 1$ the amplitude is 3×10^{-9} .

Space-Time Sampling

Space-time sampling — the interior points carry a third coordinate, t , and the whole block $\Omega \times (0, t_{\text{end}}]$ is trained at once. The solution is continuous in time and can be read at any instant, and nothing in the loss requires early times to be right before late ones.

A BLOCK, NOT A MARCH

$$(x, y, t) \in \Omega \times (0, t_{\text{end}}] \quad T(x, y, 0) = f_{IC}(x, y)$$

- **no step size** — and no stability limit
- **any instant** — read the model, do not re-run it

THE BLOCK IN PYTORCH

```
xyt = torch.rand(4000, 3)          # x, y and t together
xyt[:, 2] *= t_end                  # the time window
xy0 = torch.cat([torch.rand(400, 2), zeros], 1) # the slice at t = 0
```

$\Omega \times \text{TIME}$, SAMPLED AS ONE BLOCK



the slice one condition, few points

the block the residual, many points

Cyan is the residual, green the initial field: it is outvoted.

Initial Condition

Initial condition — the field at $t = 0$. As a loss term it is one slice among a block of interior points and is easily outvoted; built into the trial solution as $(1 - t) f_{IC} + t(\dots)\mathcal{N}$ it holds exactly, which assumes a time axis scaled to zero to one.

A TERM, OR A FACTOR

$$T = (1 - t) f_{IC}(x, y) + t x(1 - x) y(1 - y) \mathcal{N}_{\hat{w}}$$

at $t = 0$ the second term vanishes and the initial field is returned

- **soft** — three terms compete, and the slice loses
- **hard** — two terms disappear, and $t = 0$ is exact

TALK · 2 MIN How would you explain why the initial condition is the term a network loses most often, in your own words? Then, in the card on the right, what does the $(1 - t)$ factor assume, and what would break if t_{end} were ten?

THE TRIAL SOLUTION, TERM BY TERM

$(1 - t) f_{IC}$ the initial field, fading

$t \cdot x(1-x) y(1-y)$ zero at $t = 0$ and on the walls

$\mathcal{N}(x, y, t)$ free everywhere else

Each factor switches off exactly one condition.

THE START IN PYTORCH

```
L_ic = (net(xy0) - f_ic).pow(2).mean() # soft: a term
T = (1-t)*f_ic + t*Dxy*net(xyt)      # hard: a factor
```

Causality

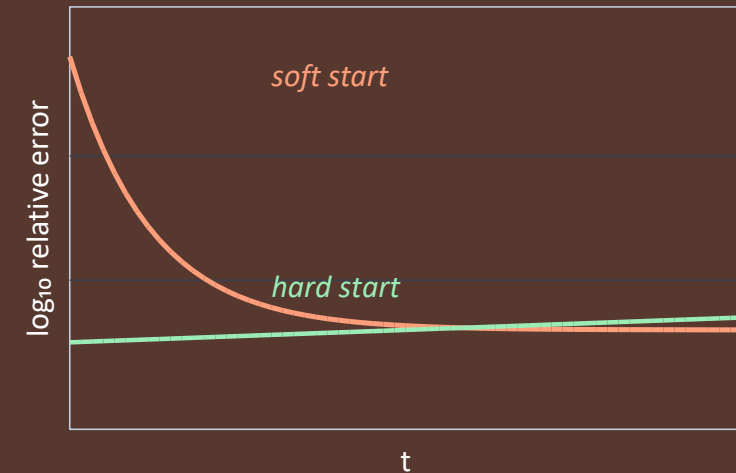
Causality — a solution at one instant follows from the earlier ones, but the loss sums over the block and treats $t = 0.01$ exactly as $t = 0.99$. A network can fit the late times first and carry an error at $t = 0$, the one instant that was known before training began.

THE ARROW OF TIME IS NOT IN THE LOSS

- **the loss** — one mean over the whole block
- **the symptom** — the largest error sits at $t = 0$
- **worse in reverse** — late times pull the early ones

TALK · 2 MIN How would you explain what causality means for a solution in time, in your own words? Then, in the plot on the right, why does the soft-start error grow towards $t = 0$, and would more interior points fix it?

SCHEMATIC · ERROR AGAINST TIME



Schematic: with a soft start the largest error sits at $t = 0$, the one instant that was known exactly before training began.

Causal Training

Causal training — four ways to put the arrow of time back into training: build the initial condition in, weight each instant by the residual accumulated before it, split the window and train the pieces in sequence, or sample the early times more densely.

WEIGHT AN INSTANT BY WHAT CAME BEFORE IT

$$w(t) = \exp\left(-\epsilon \int_0^t \mathcal{L}_{PDE}(s) ds\right)$$

a late instant counts only once the earlier residual is small

- ϵ — one more hyperparameter to set
- **the first choice** — hard-enforce, and tune nothing

CAUSAL WEIGHTS IN PYTORCH

```
res = residual(net, xyt).pow(2)           # per point, not averaged
acc = cumulative_by_time(res, xyt[:, 2])  # the residual before each t
loss = (torch.exp(-eps * acc).detach() * res).mean() # weight, then average
```

FOUR REMEDIES, IN ORDER

hard start removes the term; nothing to tune

causal weights effective; adds ϵ

time windows for long transients; more code

early points cheap; a partial fix

The order is the order to try them in. The first costs nothing, and on the problems of this block it is usually enough.

The Plate in Time

Transient plate — the plate of L8.1 with a time coordinate: uniform generation, the hole held at $T = 0$, insulated edges and a cold start. The trial solution $t\varphi\mathcal{N}$ holds the hole and the initial field at once, and the edges stay a loss term.

ONE FACTOR, TWO CONDITIONS

$$T = t \varphi(x, y) \mathcal{N}_{\hat{w}}(x, y, t)$$

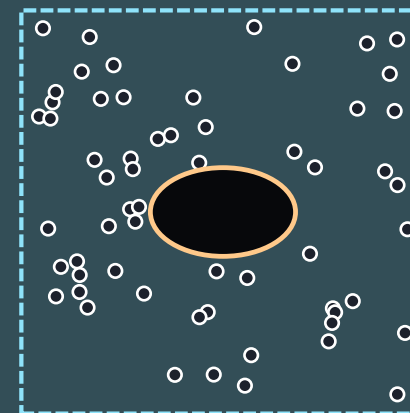
t holds the cold start, φ holds the hole: both exactly

- **built in** — the hole, and the start at zero
- **a loss term** — the insulated edges, as before

THE PLATE IN PYTORCH

```
T = t * hole_multiplier(xy) * net(xyt)    # hole and cold start, exact
F = grad(T, t) - alpha * lap(T, xy) - Q_c  # the transient residual
loss = F.pow(2).mean() + w_N * L_edges    # the edges stay a term
```

THE PLATE, NOW WITH A CLOCK



starts	cold, at $T = 0$ everywhere
loses heat	through the hole only
settles in	0.13: only the hole cools it

Verification in Time

Verification in time — a transient has no single error. The relative error at late times can be large while the absolute error is tiny, so both are reported. The flux balance holds only once the field is steady, and a time history at the hottest point catches a wrong start.

TWO ERRORS, AND TWO CHECKS WITHOUT AN ANSWER

$$\varepsilon(t) = \frac{\|T_w(\cdot, t) - T_{\text{exact}}(\cdot, t)\|_2}{\|T_{\text{exact}}(\cdot, t)\|_2}$$

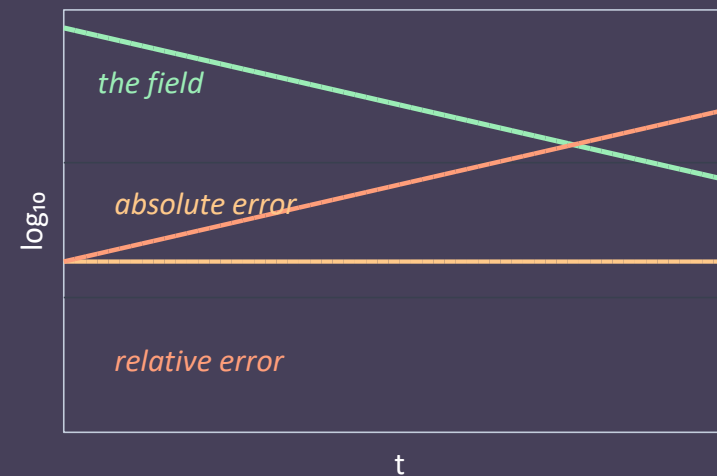
the relative error at one instant, against that instant's own size

- **report both** — relative and absolute, against t
- **without an answer** — the flux balance, and a history

ERROR AGAINST TIME IN PYTORCH

```
for t in times:
    # one instant at a time
    d = model_slice(t) - exact_slice(t) # the difference on a grid
    log(t, d.norm(), d.norm() / exact_slice(t).norm()) # absolute, relative
```

SCHEMATIC · WHY BOTH NUMBERS



The field decays, the absolute error does not, so the relative error climbs. Neither number alone says whether the answer is good.

Inverse Diffusivity

Inverse diffusivity — the diffusivity α made trainable beside the weights and identified from a measured cooling curve. It is optimised as $\log \alpha$ so that it cannot go negative, and only the early transient constrains it: once the field has decayed the data says nothing.

ONE RUN, TWO UNKNOWNNS

$$\alpha = e^\beta, \quad \mathcal{L} = \mathcal{L}_{PDE} + \mathcal{L}_{data}$$

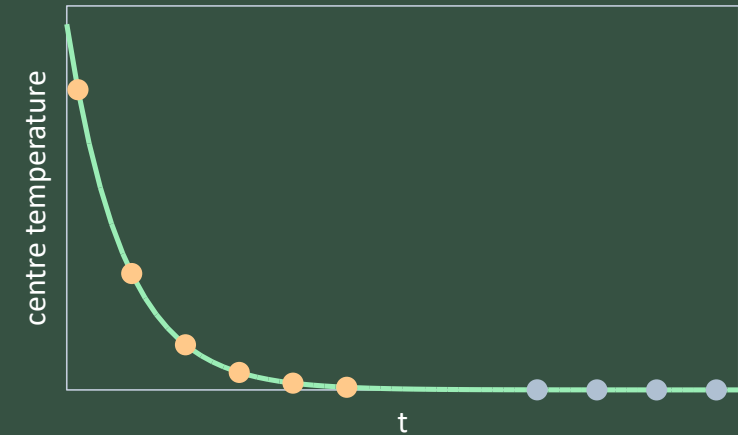
α trained as a log, and a data term beside the residual

- **the data term** — what identifies α at all
- **the window** — early, where the curve still bends

THE INVERSE IN PYTORCH

```
beta = nn.Parameter(torch.zeros(1))      # log  $\alpha$ , so  $\alpha$  stays positive
opt = torch.optim.Adam([*net.parameters(), beta]) # trained together
loss = F(net, beta.exp()).pow(2).mean() + data(T, x_s) # residual + data
```

WHERE THE DATA CARRIES α



amber, early

the curve bends: α is identified

grey, late

the curve is flat: α is free

12 sensors, $t = 0.01$ to 0.25 , noise 0.002 .

Advanced Topics

RELATED TOPICS LEFT OUT, AND WHERE THEY MATTER

topic	what it adds	read	where it makes a difference
stiffness	scales that differ by decades	Liu ch. 4.10	thin films, fast pulses
other activations	sine and swish on a transient	Liu ch. 4.7.4	oscillating sources
convergence study	error against points, in x and t	Liu ch. 4.9.3	a design sign-off
walls that change	a flux that varies with time	Liu ch. 4.6.5	a switched source

WHY THEY ARE LEFT OUT

The plate of Ex_08.2 has one timescale, a constant source and a smooth start, so none of them is needed. Each returns when a problem brings it: the switched source in a power device, the separated scales in the flow of L9.

Exercise 8.2

THE NOTEBOOKS, IN ORDER

- **00** — the timescales, the space–time sample, the reference
- **01** — the square, with a soft start
- **02** — the square, with the start built in
- **03** — the plate with its hole, in time
- **04** — the diffusivity recovered from sensors
- **05** — compares them, and the report

THE QUESTION EACH NOTEBOOK ANSWERS

- 01** where in time is the error?
- 02** what does a built-in start buy?
- 03** has the plate settled yet?
- 04** which sensors identify α ?

REPORT QUESTION, TO CONCLUDE

One loop produced the soft start, the hard start, the plate and the recovered diffusivity. What did time add to the loss, and which of your results would convince someone that the network is right?

Key Takeaways

TIME

what it is	one new term, one new input
function	the field over a block of space–time
train, tune	t_{end} from τ ; the start built in
vs steady	an initial field, and no arrow of time
used for	a device after a pulse, a cold start

PLATE AND INVERSE

what it is	the plate of L8.1, started cold
function	$t \neq N$: two conditions in one factor
train, tune	both errors; sensors early
vs forward	α trainable, found in the same run
used for	properties you cannot measure

Next — L9.1: the same loop for a flow, where the nonlinearity is the difficulty.

Questions

1 – 2 · TIME

1 What does the heat equation add to steady conduction, and how does a PINN treat time?

One time derivative, and so one initial field, with a time constant τ that says how long the transient lasts. Time is one more input: the whole block of space and time is trained at once, and the solution can be read at any instant. *Liu ch. 4.1 – 4.3; Ex_08.2 notebooks 00 – 01*

2 What is causality, and how is it put back into training?

A solution at one instant follows from the earlier ones, but the loss treats all instants alike, so the network can fit late times first and carry an error at the start. First build the initial field into the trial solution; then causal weights, a split window, or more early points. *Liu ch. 4.2 – 4.4; Ex_08.2 notebooks 01 – 02*

3 – 4 · PLATE AND INVERSE

3 How is the transient plate set up, and how is its solution checked?

The trial solution $t \phi N$ holds the initial field and the hole wall at once; the insulated edges stay a loss term. Report the relative and the absolute error, follow the hottest point in time, and use the flux balance once the field is steady. *Liu ch. 4.6; Ex_08.2 notebook 03*

4 How does an inverse problem differ from a forward one?

A parameter of the equation, here the diffusivity α , is trained beside the weights so that the solution fits measured data. It is optimised as $\log \alpha$, so it stays positive. Only the early part of the cooling curve constrains it: once the field has decayed, the data says nothing. *Liu ch. 4.6; Ex_08.2 notebook 04*

NEXT

L9.1 — Laminar Flow

The same loop again, on a velocity that carries itself: a nonlinear residual, and a pressure with no equation of its own.

BEFORE THE NEXT SESSION

read	Liu ch. 4.1 – 4.6
run	Ex_08.2 notebooks 00 to 04
bring	your error at $t = 0$, soft and hard
think about	your own problem's time constant

Block eight is finished: the plate, steady and in time, forward and inverse.